Laboratorio 4 · Algoritmos genéticos

«Ajusta tu genético»: la mochila 0/1 con diseño de experimentos

Inteligencia Artificial I · Facultad de Informática, UCM · Curso 2026-27 · Prof. Mailyn Moreno Espino

Sesión de laboratorio	Entrega	Modalidad	Material
Lunes 19 de octubre	Jueves 29 de octubre, 23:59	Por parejas	Enunciado mochila_P4.csv Ejemplo resuelto (bloques)

1. Objetivo

Implementar un algoritmo genético completo para el problema de la mochila 0/1 y encontrar, mediante un diseño de experimentos, la configuración de parámetros que lo acerque más al óptimo con un presupuesto fijo de evaluaciones.

El documento «Ejemplo resuelto: algoritmo genético y diseño de experimentos» hace este mismo proceso, completo, con el problema de los bloques. Leedlo antes de empezar: es el modelo de lo que se pide.

2. El problema

Hay n objetos; el objeto i tiene un peso w_i y un valor v_i. La mochila admite como máximo un peso C. Hay que elegir qué objetos meter para que el valor total sea máximo sin superar la capacidad:

$$\text{maximizar} \quad \sum v_i \cdot x_i \qquad \text{sujeto a} \quad \sum w_i \cdot x_i \le C, \qquad x_i \in \{0, 1\}$$

Dato	Valor
Fichero	`mochila_P4.csv` (columnas: objeto, peso, valor)
Número de objetos n	100
Capacidad C	2457
Valor óptimo	**2981** (calculado con un método exacto; no se da la solución)

Medida de calidad: gap = 100 · (2981 − valor) / 2981 %. Un gap de 0 % es el óptimo.

3. Reglas

	Regla
R1	Codificación binaria: un bit por objeto (1 = el objeto va en la mochila).
R2	El genético es vuestro: no se pueden usar librerías de algoritmos evolutivos (DEAP, PyGAD…). Sí se puede usar numpy.
R3	**Presupuesto: 2000 evaluaciones de la función de aptitud por ejecución.** Cuenta cada vez que se calcula la aptitud de un individuo, también si antes se ha reparado. La élite que pasa sin cambios no se vuelve a evaluar.
R4	Solo cuentan las soluciones que caben ($\Sigma\, w_i \cdot x_i \leq C$). La mejor solución que devuelve cada ejecución debe ser factible.
R5	30 ejecuciones por configuración, con semillas fijadas y reproducibles. Las mismas semillas para todas las configuraciones.
R6	El óptimo (2981) solo se usa para medir el gap. No puede usarse dentro del algoritmo, ni tampoco un método exacto (programación dinámica, ramificación y poda…).
R7	Las hipótesis se escriben antes de ejecutar los experimentos y se entregan al terminar la sesión del lunes 19 (Anexo A).

4. Qué hay que hacer

Fase 0 · Leer el ejemplo resuelto

Reproducid a mano, o con vuestro código, las dos generaciones del ejemplo con OneMax. Si forzáis en vuestro código los mismos torneos, cortes y mutaciones del ejemplo, debe salir exactamente la misma población; si no, hay un error.

Fase 1 · Implementación

El genético debe tener todos estos componentes, con los parámetros configurables:

Componente	Mínimo obligatorio	Parámetros
Población inicial	Aleatoria	P (tamaño de población)
Aptitud	Valor total de la mochila	—
Restricción de capacidad	**Dos tratamientos:** penalización y reparación (vosotros decidís cómo penalizar y cómo reparar)	λ (si penalizáis)
Selección	Torneo	k (tamaño del torneo)
Cruce	Un punto y uniforme	pc (probabilidad de cruce)
Mutación	Invertir bit	pm (probabilidad por bit)
Elitismo	Los e mejores pasan sin cambios (e = 0 debe ser posible)	e
Reemplazo	Generacional	—
Parada	Presupuesto de evaluaciones agotado (R3)	presupuesto = 2000

Además, el programa debe contar las evaluaciones y registrar, por ejecución: configuración, semilla, mejor valor factible, su peso y evaluaciones usadas.

Fase 2 · Hipótesis (antes de ejecutar)

Escribid al menos cinco hipótesis sobre cómo influirá cada factor en el gap, cada una con su razonamiento. Al menos una debe ser sobre el tratamiento de la restricción y otra sobre una posible interacción entre dos factores. Se entregan al final de la sesión del lunes (Anexo A) y no se pueden modificar después: en el informe se contrastan, se confirmen o no.

Fase 3 · Diseño

- Una configuración base, justificada.
- Los factores y niveles que vais a probar, y por qué esos.
- La estrategia: un factor cada vez, factorial, o una combinación de ambos. Las dos cosas a la vez tienen sentido: primero barato, para ver qué importa; después factorial donde haya dudas.
- El número total de configuraciones y de ejecuciones.

Fase 4 · Ejecución y resultados

Para cada configuración: valor medio, desviación típica, gap medio, mejor y peor valor, y número de veces que se alcanza el óptimo.

Fase 5 · Análisis

- Decidid qué diferencias son reales y cuáles son ruido, con un criterio explícito (por ejemplo, el de dos errores típicos del ejemplo resuelto).
- Contrastad cada hipótesis con los datos.
- Al menos una gráfica: por ejemplo, la evolución del mejor valor frente a las evaluaciones para tres configuraciones.

Fase 6 · Validación

Ejecutad la configuración elegida y la base con 30 semillas nuevas, que no hayáis usado para ajustar. Informad de si la mejora se mantiene.

Fase 7 · Conclusiones y configuración final

La configuración final congelada (todos sus parámetros), su gap medio en la validación y qué habéis aprendido: qué factores importan en este problema, cuáles no, y en qué se parece o se diferencia de lo que pasaba con los bloques.

5. Entregables

1. **Código (.py).** Debe reproducir la validación de la configuración final con una sola orden, por ejemplo `python p4.py --final`.
2. **Informe (PDF, máximo 8 páginas).** Fases 1 a 7, con la misma estructura que el apartado 4 del ejemplo resuelto.
3. **resultados.csv.** Una fila por ejecución: configuración, semilla, valor, peso, evaluaciones.
4. **Hoja de hipótesis (Anexo A).** Al final de la sesión del lunes 19.

6. Evaluación

Criterio	Puntos	Qué se valora
Implementación	20	Todos los componentes de la fase 1; cuenta de evaluaciones correcta; reproducibilidad; el ejemplo de OneMax sale igual.
Hipótesis y diseño	20	Hipótesis razonadas y previas; diseño justificado; coste calculado.
Rigor experimental	20	Mismas semillas, 30 réplicas, presupuesto respetado, solo soluciones factibles, validación con semillas nuevas.
Análisis	20	Criterio de ruido explícito; hipótesis contrastadas con datos; gráficas que se entienden.
Conclusiones	10	Configuración final justificada; comparación con los bloques; límites del estudio.
Calidad de la configuración final	10	Gap medio en la validación: ≤ 1 % → 10 · ≤ 2 % → 7 · ≤ 4 % → 4 · > 4 % → 1.

Anexo A · Hoja de hipótesis

Pareja: _____ Fecha: _____

	Hipótesis	Razonamiento	Cómo la comprobaremos
H1			
H2			
H3			
H4			
H5			
H6			

Anexo B · Preguntas frecuentes

Pregunta	Respuesta
¿La reparación cuenta como evaluación aparte?	No: reparar y evaluar el individuo reparado cuenta como una evaluación. Pero si evaluáis antes y después de reparar, son dos.
¿Y si una ejecución no encuentra ninguna solución factible?	Su valor es 0 y entra así en la media. Es parte del resultado de esa configuración.
¿Puedo cambiar el orden de los objetos en el cromosoma?	Sí, si lo justificáis y lo tratáis como un factor más del diseño.
¿Puedo añadir búsqueda local o una población inicial heurística?	No en la configuración final. Podéis comentarlo como extra en el informe, pero no puntúa.
¿Sirven los valores del ejemplo de los bloques?	Como punto de partida, sí. Como respuesta, no: hay que comprobarlos con la mochila.